

פרויקט גמר

מהדר לשפת CPL

קורס: קומפילציה ותרגום

שם המרצה: ד"ר רינה צביאל גרישין

מגישות: נטלי שמענוב (ת.ז: 207893496) ואור טל עדני
(ת.ז: 314669615)

תוכן עניינים

3	מבוא
4	מבנה הפרויקט
5	ארכיטקטורת המערכת
6	מבנה זרימת הבקרה במערכת
7	יכולות המהדר ומגבלותיו
7	פונקציות ראשיות במערכת
7	עקרונות תכנון בחלוקה למודולים
8	תיאור הקוד
8	מודול ראשי - cpm.c
8	מודול הניתוח הלקסיקלי - scanner.l
8	מודול הניתוח התחבירי - parser.y
8	מודול טבלת הסמלים - symtab.c, symtab.h
10	מודל יצירת קוד
10	טיפול בשגיאות מערכת
11	שימוש בכלי בינה מלאכותית במהלך מימוש הפרויקט

מבוא

בפרויקט זה מימשנו מהדר לשפת התכנות CPL. מטרת המהדר היא לקרוא קובץ קוד מקור הכתוב בשפת CPL, לבצע עליו ניתוח במספר שלבים ולבסוף להפיק קובץ listing המתאר את תהליך הקומפילציה, וכן קובץ קוד אסמבלי (MIPS).

תהליך הקומפילציה במערכת כולל מספר שלבים עיקריים:

1. ניתוח לקסיקלי - זיהוי הTokenים בקוד המקור.
2. ניתוח תחבירי - בדיקה שהמבנה התחבירי של התכנית תקין.
3. בדיקות סמנטיות - בדיקות לוגיות כגון הכרזה כפולה על משתנים, שינוי ערך של קבוע ועוד.
4. יצירת קוד - יצירת קוד אסמבלי עבור התכנית.

המהדר מומש בשפת C תוך שימוש בכלים Flex ו-Bison לצורך מימוש הניתוח הלקסיקלי והתחבירי.

מבנה הפרויקט

הפרויקט מורכב ממספר קבצים, שכל אחד מהם אחראי על חלק אחר בתהליך הקומפילציה. להלן פירוט של הקבצים בפרויקט ותפקידיהם:

שם הקובץ	תפקיד
scanner.l	מימוש הניתוח הלקסיקלי באמצעות Flex
parser.y	מימוש הניתוח התחבירי באמצעות Bison
symtab.c	מימוש טבלת הסמלים
symtab.h	הגדרות מבני הנתונים והפונקציות לטבלת הסמלים
cpm.c	תכנית ראשית המפעילה את המהדר
lex.yy.c	קוד C שנוצר אוטומטית ע"י Flex
parser.tab.c	קוד C שנוצר אוטומטית ע"י Bison
parser.tab.h	הגדרות Tokens שנוצרו אוטומטית ע"י Bison
<file_name>.cpl	תכניות קלט בשפת CPL להרצה עבור בדיקות
<file_name>.lst	קובץ פלט listing עם מספרי שורות ושגיאות
<file_name>.s	קובץ פלט שמכיל קוד אסמבלי שנוצר מהתכנית(MIPS)
readme.doc	קובץ המכיל את פרטי המגישות, הוראות ושיקולי מימוש
documentation	מסמך תיעוד זה

קבצי הbak. הם עותקי גיבוי שנוצרו במהלך הפיתוח

ארכיטקטורת המערכת

מימוש המהדר חולק למספר מודולים, כאשר כל מודול אחראי על שלב אחר בתהליך הקומפילציה. חלוקה זו מאפשרת הפרדה בין תחומי אחריות שונים במערכת, משפרת את קריאות הקוד ומקלה על תחזוקה, בדיקות והרחבות עתידיות בהמשך.

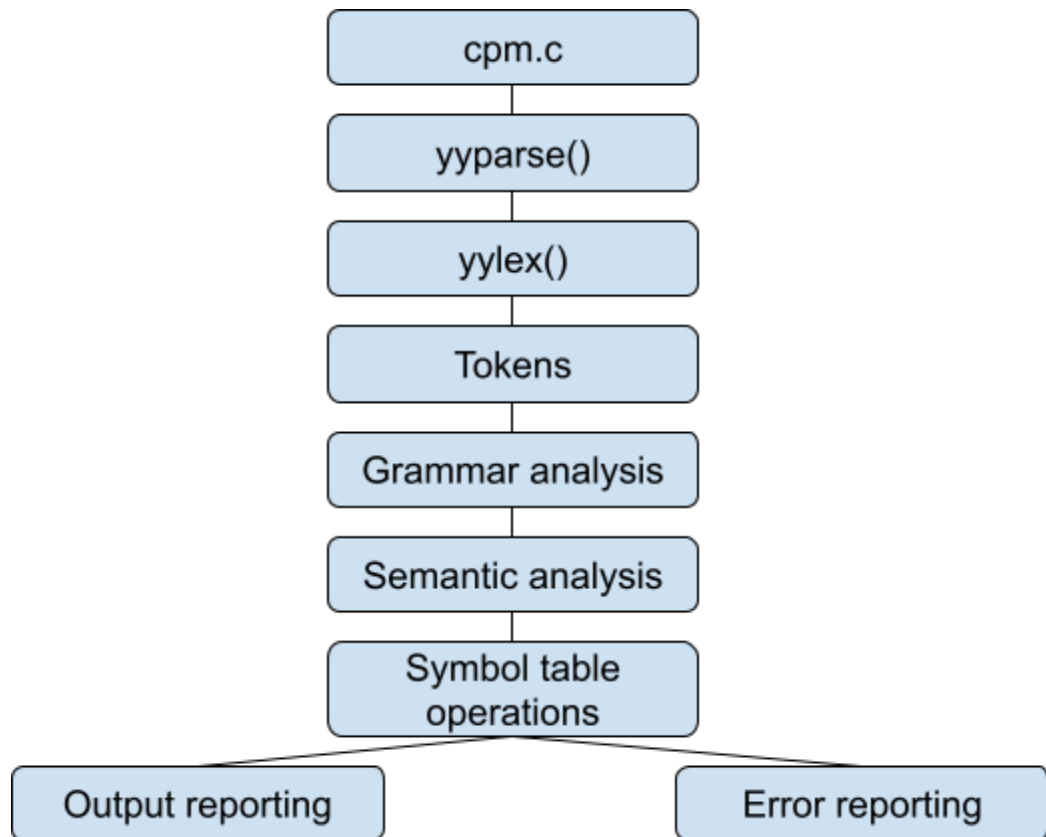
המערכת מחולקת למודולים עליהם נפרט בפרק הבא: המודול הראשי (cpm.c), מודול הניתוח הלקסיקלי (scanner.l), מודול הניתוח התחבירי (parser.y), מודול טבלת הסמלים (symtab.c+symtab.h), מודולים אוטומטיים שנוצרים במהלך הבנייה. כלל המודולים שציינו, אינם עצמאיים לחלוטין - הם עובדים בשיתוף פעולה. בפרק הבא, תיאור הקוד, נדון בקשרים בין המודולים השונים.

מבנה זרימת הבקרה במערכת

זרימת הבקרה של המהדר בנויה באופן סדרתי, כאשר כל שלב נשען על תוצאות השלב הקודם. להלן מהלך העבודה הכללי:

1. המודול הראשי נפתח ומקבל את קובץ הקלט
 2. מתבצעת פתיחה של קבצי הפלט הדרושים
 3. הפונקציה `yyparse` מופעלת
 4. במהלך עבודת הparser מתבצעות קריאות חוזרות ל`yylex` לצורך קבלת הTokens הבא
 5. הparser בודק את רצף הTokens לפי כללי הדקדוק
 6. בעת הצורך מתבצעות פעולות סמנטיות כולל גישה לטבלת הסמלים
 7. אם מתקבלות שגיאות - הן נרשמות ומדווחות (גם בCMD וגם בקובץ ה`lst`)
 8. אם הקלט תקין, נוצר קובץ קוד אסמבלי (סיומת `.s`). בנוסף, נכתב קובץ `listing` (סיומת `.lst`). המתאר את תהליך הקומפילציה.
- במקרה של שגיאה כלשהי במהלך הקומפילציה, לא נוצר קובץ קוד אסמבלי, זאת בהתאם לדרישות הפרויקט.

אפשר לתאר זאת גם כך:



יכולות המהדר ומגבלותיו

המהדר שפותח במסגרת פרויקט זה, תומך בתכונות המרכזיות של שפת CPL כפי שנדרש במסגרת המטלה. בין היכולות הנתמכות:

- הגדרת משתנים מטיפוסים שונים (int, real, string)
- ביצוע פעולות השמות וביטויים אריתמטיים
- שימוש במבני בקרה (כגון: do-til, if, while, for)
- פעולות קלט ופלט (read, print)
- בדיקות סמנטיות (כגון: הכרזה כפולה ושימוש במשתנים לא מוגדרים).

בנוסף, המהדר מפיק קוד אסמבלי תקין בשפת MIPS עבור תכניות חוקיות. עם זאת, המימוש מוגבל לתחום הדרישות של הפרויקט ואינו כולל הרחבות מעבר לכך, כגון אופטימיזציות קוד או תמיכה במבנים מורכבים נוספים שלא נדרשו במסגרת המטלה.

פונקציות ראשיות במערכת

להלן הפונקציות המרכזיות בפרויקט ותפקידן:

1. Main - פונקציית הכניסה הראשית של התכנית, נמצאת בקובץ cpm.c. אחראית על הפעלת תהליך הקומפילציה, פתיחת קבצים והעברת הבקרה ה-parser.
2. yyparse - הפונקציה הראשית של ה-parse, נוצרת ע"י Bison. תפקידה לנתח את רצף הtokens לפי כללי הדקדוק של השפה.
3. yylex - הפונקציה הראשית של ה-lexer, נוצרת ע"י Flex. תפקידה לקרוא תווים מקובץ הקלט ולהחזיר tokens לparser.
4. yyerror - פונקציה לטיפול בשגיאות תחביריות. כאשר המנתח התחבירי מזהה שגיאה תחבירית בקלט, פונקציה זו מדווחת על השגיאה. בנוסף, שגיאה זו נכתבת לקובץ listing (סיומת: .lst).
5. insertSymbol - פונקציה מתוך מודול טבלת הסמלים. מכניסה מזהה חדש לטבלה ובודקת אם קיימת רשומה עם שם זהה.
6. lookupSymbol - פונקציה מתוך מודול טבלת הסמלים. מחפשת מזהה קיים בטבלה לצורך בדיקות סמנטיות.

עקרונות תכנון בחלוקה למודולים

החלוקה למודולים בוצעה כך שכל קובץ יתמקד בתחום אחריות מוגדר:

- Scanner.i אחראי רק לניתוח לקסיקלי
- Parser.y אחראי רק לניתוח תחבירי ולפעולות סמנטיות הקשורות לדקדוק
- Symtab.c אחראי לניהול המזהים
- Cpm.c אחראי להפעלת המערכת

גישה זו מונעת ערבוב בין אחריות שונות ומאפשרת להבין את מבנה המערכת בצורה ברורה יותר. בנוסף, היא מאפשרת לבדוק כל רכיב בנפרד ולבצע שינויים במודול אחד מבלי להשפיע באופן ישיר על שאר המודולים כל עוד הממשקים ביניהם נשמרים.

תיאור הקוד

כפי שציינו קודם, מימוש המהדר חולק למספר מודולים, כאשר כל מודול אחראי על שלב אחר בתהליך הקומפילציה. כעת, לאחר שהבהרנו את ארכיטקטורת המערכת, נפרט על כל מודול ולבסוף על הקשרים בין המודולים השונים.

מודול ראשי - cpm.c

מודול זה מהווה את נקודת הכניסה של התכנית. הוא אחראי על קבלת קובץ הקלט, פתיחת קבצי העבודה והפלט, הפעלת תהליך הקומפילציה, קריאה לפונקציית yyparse וניהול תהליך הסיום והדיווח על התוצאות. זהו מודול בקרה ואינטגרציה מרכזי שאינו מבצע בעצמו ניתוח לקסיקלי או תחבירי, אלא מפעיל את שאר הרכיבים ומחבר ביניהם.

מודול הניתוח הלקסיקלי - scanner.l

מודול זה ממומש באמצעות Flex ותפקידו לסרוק את קובץ הקלט ולפרק אותו ל-Token'ים. המודול אחראי על זיהוי מזהים, מספרים, מילים שמורות ואופרטורים תוך התעלמות מרווחים, ירידות שורה והערות במקביל לעדכון מוני שורה בעת הצורך. לאחר מכן, המודול מחזיר Token'ים למודול הניתוח התחבירי באמצעות פונקציית yylex.

מודול זה מהווה את השלב הראשון בשרשרת העיבוד של הקלט.

מודול הניתוח התחבירי - parser.y

מודול זה ממומש באמצעות Bison והוא אחראי על בדיקת המבנה התחבירי של התכנית לפי דקדוק שפת CPL. המודול מקבל Token'ים מהמנתח הלקסיקלי ומנתח אותם לפי כללי הדקדוק של השפה. בנוסף, במהלך פעולות ה-Reduce מתבצעות פעולות סמנטיות הכוללות גם יצירת קוד אסמבלי (MIPS). מודול זה הוא הליבה הלוגית של המהדר מכיון שבו מתקבלת ההחלטה האם רצף ה-Token'ים המתקבל הינו מהווה תכנית חוקית.

מודול טבלת הסמלים - symtab.c, symtab.h

טבלת הסמלים מהווה רכיב מרכזי במימוש המהדר והיא משמשת לאחסון מידע על מזהים המופיעים בתכנית, כגון משתנים וקבועים.

במהלך הניתוח התחבירי והסמנטי המהדר משתמש בטבלת הסמלים כדי לוודא שמזהים הוגדרו בצורה תקינה ושלא מתבצעות טעויות כמו הכרזה כפולה או שימוש במשתנה שלא הוכרז. בחירת מבנה הנתונים לטבלת הסמלים נעשתה תוך איזון בין יעילות חיפוש לבין פשטות מימוש, בהתאם לדרישות הפרויקט.

מילים שמורות אינן מוכנסות לטבלת הסמלים, אלא מזהות בשלב הניתוח הלקסיקלי ומטופלות כ Token'ים ייחודיים.

בעת תכנון טבלת הסמלים נשקלו מספר מבני נתונים נוספים אפשריים למימוש:

- שימוש במערך סטטי

- יתרונות: גישה ישירה ומהירה לאיברים לפי אינדקס, מימוש פשוט יחסית.
- חסרונות: יש להגדיר מראש את גודל המערך, במקרה שמספר המזהים גדול מהגודל שהוגדר לא ניתן להוסיף מזהים נוספים ללא שינוי מבנה הנתונים, הקצאה גדולה מראש עלולה לבזבז זיכרון כאשר מספר המזהים קטן.
- בשל מגבלות אלו, הסקנו כי פתרון זה אינו מתאים לפרויקט שלנו, בו מספר המזהים אינו ידוע מראש.

- שימוש בטבלת גיבוב

- יתרונות: זמן חיפוש ממוצע מהיר מאוד, יעילות גבוהה כאשר יש מספר גדול של מזהים
- חסרונות: מימוש מורכב יותר, צורך בבחירת פונקציית גיבוב מתאימה, טיפול בהתנגשויות
- מאחר ומדובר בהיקף מזהים קטן יחסית בתכניות הבדיקה, שימוש בטבלת גיבוב היה מוסיף מורכבות מיותרת למימוש.

עבור מימוש טבלת הסמלים, בחרנו להשתמש במבנה נתונים של טבלת גיבוב (Hash Table) עם רשימות מקושרות לטיפול בהתנגשויות.

מבנה זה מאפשר זמן חיפוש ממוצע מהיר מאוד ($O(1)$), תוך שמירה על גמישות בהוספת מזהים חדשים במהלך ריצה.

כל תא בטבלה מצביע לרשימה מקושרת של סמלים בעלי אותו ערך גיבוב, כך שניתן לטפל בהתנגשויות בצורה יעילה ופשוטה.

הבחירה במבנה זה נעשתה מתוך רצון לאזן בין יעילות חיפוש לבין פשטות מימוש ומתאימה גם כאשר מספר המזהים בתכנית אינו ידוע מראש.

מכיון שמספר המזהים בתכנית אינו ידוע מראש, כל רשומה בטבלת הסמלים מוקצית בזיכרון באופן דינמי באמצעות פונקציית malloc.

הטבלה אינה מוגבלת בגודלה, והיא מתרחבת דינמית במהלך הריצה בהתאם לצורך. בעת הכנסת מזהה חדש לטבלה מתבצעת הקצאת זיכרון עבור רשומה חדשה ולאחר מכן מתבצע אתחול של שדות הרשומה והוספתה לרשימה המקושרת.

שימוש בהקצאת זיכרון דינמית מאפשר הגדלה הדרגתית של הטבלה לפי צורך, ניצול יעיל של הזיכרון וגמישות במבנה הנתונים.

בנוסף, כל רשומה מקושרת לאיבר הבא באמצעות מצביע next, דבר המאפשר מעבר על כל טבלת הסמלים בעת ביצוע פעולות חיפוש או בדיקות סמנטיות.

המערכת מממשת מספר פעולות מרכזיות על טבלת הסמלים, המרכזיות מביניהן הן הוספת מזהה חדש לטבלה, חיפוש מזהה בטבלה ובדיקת הכרזה כפולה. באמצעות פעולות אלו ניתן לבצע בדיקות סמנטיות במהלך תהליך הקומפילציה.

מודל יצירת קוד

במהדר זה, תהליך יצירת הקוד מתבצע במהלך הניתוח התחבירי באמצעות פעולות סמנטיות המשולבות בכללי הדקדוק ב-Bison.

ביטויים אריתמטיים מחושבים באופן רקורסיבי, כאשר ערכים זמניים נשמרים ברגיסטים ($t0$, $t1$) עבור מספרים שלמים ו- $f0$, $f1$) עבור מספרים ממשיים, במקרה הצורך נעשה שימוש במחסנית (sp) לשמירת תוצאות ביניים.

פקודות בסיסיות מתורגמות באופן ישיר לקוד MIPS באופן הבא:

פעולת השמה מתורגמת לטעינת ערך לרגיסטר ולאחר מכן כתיבה לזיכרון.

פעולות קלט/פלט משתמשות ב-syscall המתאים (לדוגמה: 1 עבור הדפסת מספר שלם, 2 עבור מספר ממשי, 4 עבור הדפסת מחרוזת, 5/6 עבור הדפסת קלט).

מבני בקרה כגון if, while, for ו-momמים באמצעות labels וקפיצות (branch instructions). לכל מבנה בקרה נוצרים labels ייחודיים בעזרת פונקציה ייעודית, והזרימה מתבצעת באמצעות פקודות קפיצה מותנות ולא מותנות. ביטויים בוליאניים מחושבים כך שהתוצאה נשמרת ברגיסטר $t0$ כ-0 או 1. עבור מספרים שלמים נעשה שימוש בפקודות השוואה כמו slt, ואילו עבור מספרים ממשיים נעשה שימוש בפקודות נקודה צפה כגון c.lt.s בשילוב עם פקודות קפיצה.

בנוסף, במקרה של ביטויים עם טיפוסים שונים, מבוצעת המרה מתאימה (למשל int ל-float) באמצעות פקודות כמו cvt.s.w-mtc1, בהתאם לכללי השפה.

מודל זה מאפשר תרגום ישיר ויעיל של תוכניות CPL לקוד MIPS תקין הניתן להרצה תקינה בסביבת MARS.

טיפול בשגיאות מערכת

המהדר תומך בטיפול במספר סוגי שגיאות במהלך תהליך הקומפילציה:

- שגיאות לקסיקליות - מתרחשות כאשר מזהה תו או רצף תווים שאינו תואם לאף Token חוקי בשפה. במקרים אלו, מדווחת שגיאה עם מספר השורה הרלוונטי.
- שגיאות תחביריות - מתגלות במהלך הניתוח התחבירי כאשר רצף ה-Tokenים אינו תואם את כללי הדקדוק. במצב זה מופעלת הפונקציה yyerror, אשר מדווחת על מיקום השגיאה וסוג הבעיה.
- שגיאות סמנטיות - כוללות מקרים כגון שימוש במשתנה שלא הוגדר, הכרזה כפולה על משתנים או ניסיון לשנות ערך של קבוע. שגיאות אלו נבדקות במהלך הפעולות הסמנטיות תוך שימוש בטבלת הסמלים. במקרה של שגיאה, המערכת מדווחת עליה גם למסך וגם לקובץ ה-lst, תוך שמירה על המשך ניתוח התכנית ככל האפשר (error recovery), במטרה לאתר שגיאות נוספות באותה הרצה.

חשוב לציין כי במקרה בו זוהתה שגיאה כלשהי במהלך הקומפילציה, לא ייוצר קובץ פלט של קוד האסמבלי (.s) בהתאם לדרישות הפרויקט.

שימוש בכלי בינה מלאכותית במהלך מימוש הפרויקט

במהלך מימוש הפרויקט נעשה שימוש בכלי בינה מלאכותית (AI) כגון Gemini ו-ChatGPT. כלים אלו שימשו בעיקר לפתרון בעיות טכניות ובאגים שצצו לאורך הדרך, ובנוסף, כעוזרי קוד וייעוץ טכני.

- בשלב הניתוח הלקסיקלי נעשה שימוש בכלי AI לצורך הבנת מבנה חוקי הביטויים וכתיבת ביטויים רגולריים מתאימים.
- בשלב הניתוח התחבירי נעזרנו בכלי AI לצורך הבנת מבנה הדקדוק, זיהוי קונפליקטים ופתרונם.
- בשלב הסמנטי ויצירת הקוד נעשה שימוש בכלי AI לצורך פתרון בעיות הקשורות לטיפוסים, המרות ויצירת קוד MIPS תקין.